



ETS  
Ingeniería de  
Telecomunicación

UNIVERSIDAD POLITÉCNICA DE CARTAGENA

ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA DE TELECOMUNICACIÓN

Máster Universitario en Ingeniería de Telecomunicación

---

# GEMEROTIC: Arquitectura de un Constructor de Gemelos Digitales para Ciberseguridad en Redes OT/IT



Universidad  
Politécnica  
de Cartagena

TRABAJO FIN DE MÁSTER

Autor:

**Enrique Moreno**

Directores:

**Pendiente de confirmar**

CARTAGENA, 2026

Las redes industriales OT que soportan procesos productivos y servicios esenciales se han convertido en un objetivo prioritario de la ciberseguridad moderna. Su protección no puede abordarse del mismo modo que una red IT convencional: la disponibilidad de la planta, la seguridad física, los equipos heredados, las ventanas de mantenimiento reducidas y la convivencia de protocolos industriales con tecnologías IP hacen que cualquier auditoría intrusiva, prueba de penetración o cambio de configuración sobre producción pueda generar impactos operativos relevantes.

Este Trabajo Fin de Máster se plantea en colaboración con AMC Global, grupo empresarial español del sector de alimentación y bebidas naturales con infraestructuras IT y OT distribuidas en varios centros productivos. La colaboración aporta un contexto industrial realista: una organización con procesos de producción continuos, necesidades de inventario técnico, exigencias de ciber-resiliencia y obligaciones crecientes vinculadas al Esquema Nacional de Seguridad, la Directiva NIS2 y marcos de referencia como ISA/IEC 62443 y NIST SP 800-82.

La propuesta de valor del trabajo consiste en explorar la viabilidad de aplicar gemelos digitales a la ciberseguridad de redes OT. Para ello se diseña la arquitectura de GEMEROTIC, un constructor de gemelos digitales orientado a redes industriales. La idea no es crear únicamente un diagrama de red, sino establecer las bases de una herramienta de modelado técnico, comparable conceptualmente a un entorno de diseño asistido, capaz de representar activos, ubicaciones, conexiones, segmentación, configuración y evidencias de validación.

La arquitectura propuesta separa la información en tres capas: infraestructura física, conectividad lógica y seguridad OT. Sobre este modelo se construye un flujo que parte de una interfaz visual, persiste el estado editable del proyecto, genera inventario enriquecido, produce artefactos reproducibles y permite desplegar o validar el gemelo en un entorno controlado. NetBox se emplea como repositorio derivado de inventario técnico, mientras que PostgreSQL mantiene el estado granular del gemelo dentro de GEMEROTIC. El pipeline genera artefactos para herramientas como Containerlab, Ansible, Batfish y OPA, incorporando además una capa inicial de cumplimiento

---

determinista y asistencia explicativa basada en IA.

El resultado esperado es una base sólida para un constructor de gemelos digitales aplicable a redes OT, con capacidad para reproducir topologías existentes, analizar cambios antes de aplicarlos en planta, apoyar auditorías técnicas y facilitar procesos de mejora continua sin interrumpir la operación real. El trabajo no pretende certificar automáticamente el cumplimiento normativo ni sustituir una auditoría formal, sino demostrar una arquitectura viable para estructurar evidencias, reducir riesgo operativo y evolucionar hacia validaciones más avanzadas.

---

## ABSTRACT

Industrial OT networks supporting production processes and essential services have become a major cybersecurity concern. Their protection cannot be approached in the same way as a conventional IT network: plant availability, physical safety, legacy equipment, limited maintenance windows and the coexistence of industrial protocols with IP technologies make intrusive audits, penetration testing or configuration changes on production systems potentially disruptive.

This Master's Thesis is developed in collaboration with AMC Global, a Spanish business group in the natural food and beverage sector with distributed IT and OT infrastructures across several production sites. This collaboration provides a realistic industrial context: an organisation with continuous production processes, technical inventory needs, cyber-resilience requirements and increasing obligations related to the Spanish National Security Framework, the NIS2 Directive and reference frameworks such as ISA/IEC 62443 and NIST SP 800-82.

The value proposition of this work is to explore the feasibility of applying digital twins to OT network cybersecurity. To this end, the thesis designs the architecture of GEMEROTIC, a digital twin constructor focused on industrial networks. The objective is not merely to create a network diagram, but to establish the foundations of a technical modelling tool, conceptually similar to a computer-aided design environment, able to represent assets, locations, connections, segmentation, configuration and validation evidence.

The proposed architecture separates information into three layers: physical infrastructure, logical connectivity and OT security. Based on this model, the system starts from a visual interface, persists the editable project state, generates enriched inventory, produces reproducible artefacts and enables the digital twin to be deployed or validated in a controlled environment. NetBox is used as a derived technical inventory repository, while PostgreSQL stores the granular state of the twin within GEMEROTIC. The pipeline generates artefacts for tools such as Containerlab, Ansible, Batfish and OPA, also incorporating an initial deterministic compliance layer and AI-assisted explanation.

---

The expected result is a solid foundation for a digital twin constructor applicable to OT networks, capable of reproducing existing topologies, analysing changes before applying them to plant environments, supporting technical audits and enabling continuous improvement without interrupting real operations. The work does not aim to automatically certify regulatory compliance or replace a formal audit, but to demonstrate a viable architecture for structuring evidence, reducing operational risk and evolving towards more advanced validations.

---

## ÍNDICE GENERAL

|                                                         |           |
|---------------------------------------------------------|-----------|
| <b>Resumen</b>                                          | <b>2</b>  |
| <b>Abstract</b>                                         | <b>4</b>  |
| <b>1. Introducción</b>                                  | <b>13</b> |
| 1.1. Contexto y motivación . . . . .                    | 13        |
| 1.2. Problema abordado . . . . .                        | 14        |
| 1.3. Objetivo general . . . . .                         | 14        |
| 1.4. Objetivos específicos . . . . .                    | 14        |
| 1.5. Alcance del trabajo . . . . .                      | 15        |
| 1.6. Estructura del documento . . . . .                 | 15        |
| <b>2. Estado del Arte y Marco Normativo</b>             | <b>16</b> |
| 2.1. Redes industriales OT/IT . . . . .                 | 16        |
| 2.2. Particularidades de la ciberseguridad OT . . . . . | 16        |

|                                                                 |           |
|-----------------------------------------------------------------|-----------|
| 2.3. Modelos de referencia para entornos industriales . . . . . | 17        |
| 2.4. Normativas y estándares aplicables . . . . .               | 17        |
| 2.5. Gemelos digitales aplicados a ciberseguridad . . . . .     | 17        |
| 2.6. Herramientas relacionadas . . . . .                        | 18        |
| 2.7. Limitaciones de las aproximaciones existentes . . . . .    | 18        |
| <b>3. Metodología y Diseño del Trabajo</b>                      | <b>19</b> |
| 3.1. Enfoque metodológico . . . . .                             | 19        |
| 3.2. Requisitos funcionales . . . . .                           | 19        |
| 3.3. Requisitos no funcionales . . . . .                        | 20        |
| 3.4. Decisiones de diseño principales . . . . .                 | 20        |
| 3.5. Estrategia visual de la memoria . . . . .                  | 21        |
| 3.6. Criterios de validación . . . . .                          | 21        |
| <b>4. Arquitectura de GEMEROTIC</b>                             | <b>23</b> |
| 4.1. Visión general . . . . .                                   | 23        |
| 4.2. Principios arquitectónicos . . . . .                       | 23        |
| 4.3. Modelo de datos en tres capas . . . . .                    | 24        |
| 4.4. Constructor visual de topologías . . . . .                 | 25        |
| 4.5. Backend, API y persistencia granular . . . . .             | 25        |
| 4.6. Inventario enriquecido con NetBox . . . . .                | 26        |
| 4.7. Generación de artefactos . . . . .                         | 26        |

|                                                             |           |
|-------------------------------------------------------------|-----------|
| 4.8. Runtime transparente del gemelo . . . . .              | 26        |
| 4.9. Validación y cumplimiento . . . . .                    | 27        |
| 4.10. Estado de implementación de la arquitectura . . . . . | 27        |
| <b>5. Implementación del Prototipo</b>                      | <b>29</b> |
| 5.1. Estructura del repositorio . . . . .                   | 29        |
| 5.2. Implementación del modelo de datos . . . . .           | 29        |
| 5.3. Implementación del frontend . . . . .                  | 30        |
| 5.4. Implementación del backend . . . . .                   | 30        |
| 5.5. Implementación del pipeline . . . . .                  | 30        |
| 5.6. Implementación del cumplimiento . . . . .              | 31        |
| 5.7. Despliegue y entorno de ejecución . . . . .            | 31        |
| <b>6. Validación y Resultados</b>                           | <b>32</b> |
| 6.1. Estrategia de pruebas . . . . .                        | 32        |
| 6.2. Validación del modelo de datos . . . . .               | 32        |
| 6.3. Validación del constructor visual . . . . .            | 33        |
| 6.4. Validación de persistencia e inventario . . . . .      | 33        |
| 6.5. Validación del pipeline y runtime . . . . .            | 33        |
| 6.6. Validación de cumplimiento . . . . .                   | 34        |
| 6.7. Resultados obtenidos . . . . .                         | 34        |
| 6.8. Limitaciones observadas . . . . .                      | 35        |

|                                                            |           |
|------------------------------------------------------------|-----------|
| <b>7. Análisis del Valor y Aplicabilidad Industrial</b>    | <b>36</b> |
| 7.1. Valor para redes OT reales . . . . .                  | 36        |
| 7.2. Comparación frente a enfoques tradicionales . . . . . | 36        |
| 7.3. Aplicabilidad a sectores críticos . . . . .           | 37        |
| 7.4. Encaje con estándares y auditorías . . . . .          | 37        |
| 7.5. Evolución empresarial . . . . .                       | 37        |
| <b>8. Conclusiones y Trabajo Futuro</b>                    | <b>39</b> |
| 8.1. Conclusiones técnicas . . . . .                       | 39        |
| 8.2. Conclusiones de ciberseguridad OT . . . . .           | 39        |
| 8.3. Limitaciones del prototipo . . . . .                  | 40        |
| 8.4. Trabajo futuro . . . . .                              | 40        |
| <b>Bibliografía</b>                                        | <b>41</b> |
| <b>Anexos</b>                                              | <b>43</b> |
| <b>A. Manual de instalación y despliegue</b>               | <b>44</b> |
| <b>B. Ejemplos de payload TopologyCreate</b>               | <b>45</b> |
| <b>C. Artefactos generados</b>                             | <b>46</b> |
| <b>D. Capturas de interfaz y resultados</b>                | <b>47</b> |
| <b>E. Código de interés</b>                                | <b>48</b> |

ÍNDICE DE FIGURAS

|                                                                  |    |
|------------------------------------------------------------------|----|
| 4.1. Flujo arquitectónico global de GEMEROTIC. . . . .           | 23 |
| 4.2. Transformación del estado visual en modelo técnico. . . . . | 25 |

---

## ÍNDICE DE TABLAS

|                                                                                      |    |
|--------------------------------------------------------------------------------------|----|
| 4.1. Modelo de datos en tres capas empleado por GEMEROTIC. . . . .                   | 24 |
| 4.2. Artefactos derivados del modelo GEMEROTIC. . . . .                              | 26 |
| 4.3. Estado de implementación de los módulos principales de GEMEROTIC. . . . .       | 27 |
| 6.1. Evidencias de persistencia e inventario para el proyecto de validación. . . . . | 33 |

---

## ÍNDICE DE CÓDIGOS

## 1.1. Contexto y motivación

La digitalización de la industria ha incrementado la interdependencia entre las redes corporativas IT y las redes industriales OT. En una planta moderna conviven sistemas de planificación, servidores, estaciones de ingeniería, controladores, HMI, PLC, sensores, actuadores, comunicaciones industriales y servicios conectados. Esta convergencia facilita la supervisión y la eficiencia operativa, pero también amplía la superficie de exposición frente a incidentes de ciberseguridad.

El trabajo se contextualiza en una colaboración con AMC Global, grupo empresarial español especializado en el sector de alimentación y bebidas naturales. La compañía desarrolla actividad industrial con múltiples centros de producción e infraestructuras IT y OT relevantes para la continuidad del negocio. La información pública de la compañía describe un grupo global orientado a la producción y comercialización de productos naturales, con plantas internacionales, una fuerte actividad de innovación y un volumen significativo de producción y referencias comerciales [1, 2]. En el marco del trabajo, AMC Global se considera un escenario industrial de referencia para estudiar cómo un gemelo digital puede apoyar la documentación, análisis y mejora continua de redes OT.

El punto de partida es común en muchas organizaciones industriales: las redes OT son críticas para la operación, pero no siempre existe una documentación completa, actualizada y conectada con criterios de ciberseguridad. Además, las auditorías intrusivas, el pentesting o la modificación directa de configuraciones sobre producción pueden suponer un riesgo operativo. Un error en una regla de firewall, una actualización no verificada o una prueba de escaneo agresiva pueden afectar a la disponibilidad de una línea de producción. Por ello, resulta necesario disponer de entornos seguros donde diseñar, reproducir y validar cambios antes de actuar sobre la instalación real.

## 1.2. Problema abordado

El problema que aborda este trabajo es la dificultad de modelar, inventariar y validar redes OT de forma estructurada sin interferir en la operación real. Aunque existen herramientas maduras para inventario, automatización o simulación de redes, muchas de ellas parten de entornos bien documentados, redes modernas o necesidades propias de IT. En cambio, las redes OT suelen incorporar equipos legacy, protocolos industriales, restricciones de disponibilidad, segmentación por zonas y conductos, y requisitos normativos específicos.

La hipótesis de partida es que un constructor de gemelos digitales orientado a redes OT puede reducir esta brecha. La herramienta debe permitir dibujar y editar la red como si se tratara de un entorno de diseño asistido, pero su resultado debe ser un modelo técnico formal, no un diagrama aislado. Ese modelo debe poder transformarse en inventario, artefactos reproducibles, despliegues de laboratorio y evidencias de validación.

## 1.3. Objetivo general

El objetivo general del Trabajo Fin de Máster es diseñar la arquitectura y establecer las bases de un constructor de gemelos digitales para redes industriales OT/IT, capaz de representar activos, conectividad, segmentación y configuración de una red OT, generar inventario técnico y facilitar validaciones de ciberseguridad en un entorno seguro y controlado.

## 1.4. Objetivos específicos

Para alcanzar el objetivo general se plantean los siguientes objetivos específicos:

1. Analizar las necesidades de ciberseguridad, inventario y mejora continua en redes industriales OT, tomando como referencia el contexto de colaboración con AMC Global.
2. Estudiar los marcos normativos y técnicos aplicables a redes OT, especialmente ISA/IEC 62443, NIST SP 800-82, NIS2 y el Esquema Nacional de Seguridad.
3. Diseñar un modelo de datos que separe infraestructura física, conectividad lógica y seguridad OT, incorporando activos, puertos, cables, interfaces, VLAN, zonas, conductos, niveles Purdue y niveles de seguridad.
4. Definir la arquitectura de GEMEROTIC como constructor visual de gemelos digitales, diferenciando el estado editable del proyecto, el inventario derivado y los artefactos de validación.
5. Implementar y documentar un prototipo funcional que permita persistir el estado del gemelo, sincronizar inventario técnico, generar artefactos reproducibles y reconciliar un runtime de

laboratorio.

6. Validar la arquitectura mediante pruebas automatizadas y evidencias por capa: modelo, API, frontend, persistencia granular, inventario NetBox, artefactos, Containerlab, Ansible y consola runtime.
7. Identificar limitaciones y líneas de evolución hacia auditoría avanzada, RAG normativo, observabilidad y validación más profunda de configuraciones de red.

## 1.5. Alcance del trabajo

El alcance del trabajo se centra en la arquitectura y prototipo de un constructor de gemelos digitales para redes OT. La memoria no se redacta como una bitácora de desarrollo por pasos, sino como una propuesta de ingeniería aplicada: se describe el problema industrial, se justifican las decisiones de diseño, se explica la arquitectura del sistema y se presentan evidencias de validación.

El prototipo no pretende sustituir una auditoría formal ni certificar automáticamente el cumplimiento de una organización. Su finalidad es crear una representación técnica controlada que permita documentar, analizar y ensayar cambios con menor riesgo. Las capacidades de cumplimiento se plantean como apoyo a la auditoría y a la mejora continua, no como decisión legal autónoma.

## 1.6. Estructura del documento

El capítulo 2 presenta el estado del arte y el marco normativo asociado a redes OT, gemelos digitales, inventario y validación de ciberseguridad. El capítulo 3 describe la metodología del trabajo, los requisitos y las decisiones de diseño. El capítulo 4 desarrolla la arquitectura de GEMEROTIC. El capítulo 5 resume la implementación del prototipo. El capítulo 6 recoge la estrategia de validación y los resultados previstos. El capítulo 7 analiza la aplicabilidad industrial del enfoque. Finalmente, el capítulo 8 presenta las conclusiones y líneas de trabajo futuro.

## CAPÍTULO 2

### ESTADO DEL ARTE Y MARCO NORMATIVO

#### 2.1. Redes industriales OT/IT

Las redes OT integran sistemas encargados de monitorizar o modificar procesos físicos: PLC, HMI, SCADA, RTU, sensores, actuadores, estaciones de ingeniería, sistemas de historización y redes de planta. A diferencia de una red IT convencional, su objetivo principal no es únicamente proteger información, sino preservar la continuidad, seguridad y disponibilidad de procesos industriales.

La convergencia IT/OT permite centralizar datos, optimizar la producción y conectar plantas con servicios corporativos, pero también introduce dependencias nuevas. Un incidente que en IT podría limitarse a pérdida de confidencialidad o indisponibilidad temporal puede traducirse en OT en parada de línea, pérdida de producción, afectación de seguridad física o incumplimiento regulatorio. Por ello, la arquitectura de red, la segmentación y la trazabilidad de cambios son elementos esenciales.

#### 2.2. Particularidades de la ciberseguridad OT

La ciberseguridad OT presenta restricciones específicas. Los equipos tienen ciclos de vida largos, pueden ejecutar firmware antiguo, no siempre admiten agentes de seguridad y suelen operar con protocolos diseñados antes de que la autenticación o el cifrado fueran requisitos comunes. Además, muchas instalaciones industriales no pueden detenerse con facilidad para aplicar cambios o ejecutar pruebas.

Esta realidad hace que las auditorías técnicas deban plantearse con precaución. Un escaneo ac-

tivo, una prueba de explotación o una actualización no ensayada puede generar comportamientos inesperados. NIST SP 800-82 destaca precisamente que la tecnología operacional debe protegerse considerando requisitos particulares de rendimiento, fiabilidad y seguridad física [3]. En este contexto, un gemelo digital aporta un entorno intermedio para documentar, ensayar y validar antes de intervenir sobre producción.

## 2.3. Modelos de referencia para entornos industriales

El modelo Purdue y la norma ISA-95 se utilizan como referencias para ordenar la arquitectura funcional de una instalación industrial. Aunque no son modelos de seguridad por sí mismos, ayudan a diferenciar niveles de campo, control, supervisión, operaciones, DMZ industrial y red corporativa. Esta separación facilita analizar rutas de comunicación, dependencias entre dominios y exposición de activos críticos.

ISA/IEC 62443 aporta conceptos directamente aplicables a la ciberseguridad OT, especialmente zonas, conductos y niveles de seguridad [4]. Una zona agrupa activos con requisitos de seguridad similares, mientras que un conducto define comunicaciones permitidas entre zonas. Estos conceptos encajan de forma natural con un gemelo digital, porque convierten la segmentación en un objeto modelable, auditable y validable.

## 2.4. Normativas y estándares aplicables

La Directiva NIS2 refuerza en la Unión Europea las obligaciones de gestión de riesgos, continuidad, supervisión y notificación de incidentes para entidades esenciales e importantes [5]. En España, el Esquema Nacional de Seguridad establece principios básicos y requisitos mínimos orientados a proteger información y servicios, incluyendo gestión basada en riesgos, prevención, detección, respuesta, vigilancia continua y reevaluación periódica [6].

Estos marcos no convierten automáticamente una herramienta técnica en una solución de cumplimiento, pero sí proporcionan una guía para estructurar evidencias. En el caso de GEMEROTIC, las referencias normativas sirven para justificar qué información debe recoger el modelo, qué relaciones son relevantes y cómo pueden presentarse hallazgos técnicos de forma trazable.

## 2.5. Gemelos digitales aplicados a ciberseguridad

Un gemelo digital aplicado a ciberseguridad debe ir más allá de una representación visual. Para aportar valor en redes OT, debe mantener una relación explícita entre activos, ubicaciones, puertos, cableado, interfaces, VLAN, configuración, zonas, conductos y evidencias de validación. Solo así

puede convertirse en una herramienta útil para análisis de cambios, preparación de auditorías y mejora continua.

En este trabajo se adopta el concepto de constructor de gemelos digitales como un entorno de diseño técnico. El usuario puede trabajar visualmente, pero el sistema debe generar un modelo formal persistente. La analogía con una herramienta tipo AutoCAD resulta útil: el lienzo permite diseñar, pero el valor real reside en que lo dibujado tiene estructura, propiedades y capacidad de derivar artefactos técnicos.

## 2.6. Herramientas relacionadas

NetBox se utiliza en el sector como plataforma de inventario de red e IPAM/DCIM [7]. Containerlab permite definir laboratorios de red basados en contenedores [8]. Ansible facilita la automatización declarativa de configuraciones [9]. Batfish proporciona análisis estático de configuraciones de red [10]. OPA permite expresar políticas como código [11]. FastAPI, Pydantic, Jinja2 y React Flow aportan la base software del prototipo [12, 13, 14, 15].

La aportación de GEMEROTIC no consiste en sustituir estas herramientas, sino en integrarlas en una arquitectura orientada a redes OT. El modelo de tres capas y la persistencia granular permiten coordinar inventario, artefactos y runtime desde una representación común.

## 2.7. Limitaciones de las aproximaciones existentes

Muchas soluciones existentes abordan una parte del problema: inventario, automatización, simulación, análisis de configuración o cumplimiento. Sin embargo, en una organización industrial el reto suele estar en conectar esas piezas sin asumir que la red está perfectamente documentada ni que puede modificarse sin riesgo.

GEMEROTIC se sitúa en ese hueco: prioriza la construcción del conocimiento técnico de la red, la generación de inventario enriquecido y la validación en laboratorio antes de cualquier cambio sobre producción. Esta decisión encaja especialmente con organizaciones industriales que necesitan avanzar en ciber-resiliencia y conformidad normativa sin comprometer la continuidad operacional.

## CAPÍTULO 3

---

## METODOLOGÍA Y DISEÑO DEL TRABAJO

### 3.1. Enfoque metodológico

El trabajo adopta una metodología de ingeniería aplicada. En primer lugar se analiza el problema industrial: redes OT críticas, dificultad de inventario, riesgo de pruebas sobre producción y necesidad de mejora continua. A continuación se estudian los marcos técnicos y normativos relevantes. Finalmente se diseña e implementa un prototipo modular que permite validar la viabilidad del enfoque.

La colaboración con AMC Global orienta la memoria hacia un contexto realista. No se busca construir una solución cerrada para una única planta, sino definir una arquitectura replicable que pueda adaptarse a entornos productivos con múltiples sedes, activos legacy, segmentación industrial y necesidades de auditoría. Por ello, la metodología combina análisis normativo, diseño de arquitectura software, prototipado incremental y validación por evidencias.

### 3.2. Requisitos funcionales

Los requisitos funcionales principales son:

- RF-1. Permitir al usuario construir visualmente una topología OT/IT con activos industriales y de red.
- RF-2. Representar infraestructura física, conectividad lógica y seguridad OT como capas diferenciadas.

- RF-3. Persistir el estado editable del gemelo para evitar pérdida de trabajo ante fallos de integraciones externas.
- RF-4. Generar inventario técnico derivado en NetBox, incluyendo dispositivos, interfaces, cables, VLAN, zonas y conductos.
- RF-5. Producir artefactos reproducibles para Containerlab, Ansible, Batfish y políticas de cumplimiento.
- RF-6. Reconciliar un runtime de laboratorio de forma transparente a partir del último estado persistido.
- RF-7. Permitir interacción operativa con nodos del laboratorio, incluyendo consola, acciones de encendido/apagado y sincronización de configuración cuando proceda.
- RF-8. Generar informes de cumplimiento técnico con evidencias y asistencia explicativa limitada al informe calculado.

### 3.3. Requisitos no funcionales

La arquitectura debe cumplir requisitos no funcionales especialmente relevantes en OT:

- **Trazabilidad:** cada entidad del gemelo debe poder relacionarse con evidencias, artefactos o inventario.
- **Reproducibilidad:** los laboratorios y configuraciones deben generarse de forma determinista desde el modelo.
- **Separación de responsabilidades:** la capa física, lógica y de seguridad no deben mezclarse.
- **Tolerancia a fallos externos:** un error de NetBox, runtime o validación no debe destruir el estado editable del usuario.
- **Seguridad operacional:** las acciones sobre runtime deben estar controladas, autenticadas y limitadas.
- **Extensibilidad:** el diseño debe admitir módulos futuros como observabilidad, auditoría OPA avanzada o RAG normativo.

### 3.4. Decisiones de diseño principales

La primera decisión es utilizar un modelo canónico propio de GEMEROTIC. NetBox se integra como inventario enriquecido derivado, pero no gobierna el estado operativo del gemelo. Esta distinción evita bloquear la edición cuando una integración externa falla y permite mantener el foco en la topología validable.

La segunda decisión es persistir el proyecto en PostgreSQL granular. El estado visual, las entidades físicas, lógicas, de seguridad y la configuración por activo se almacenan como registros independientes. Esta estructura permite evolucionar desde el snapshot inicial hacia comandos granulares, revisiones y eventos de outbox.

La tercera decisión es emplear generación declarativa de artefactos. En lugar de ejecutar cambios directos sobre producción, el sistema produce ficheros inspeccionables que pueden alimentar laboratorios, automatización y validaciones. Esta aproximación es coherente con el objetivo de reducir riesgo en redes OT.

La cuarta decisión es separar cumplimiento determinista y asistencia IA. El motor de cumplimiento calcula hallazgos a partir de evidencias técnicas; la IA solo explica y prioriza esos resultados. Así se evita presentar un modelo generativo como autoridad normativa.

### 3.5. Estrategia visual de la memoria

La memoria se apoyará en tres tipos de material visual. En primer lugar, diagramas conceptuales generados con Mermaid o Excalidraw para explicar arquitectura, modelo de datos, pipeline y flujo de persistencia. En segundo lugar, tablas de trazabilidad inspiradas en los TFM de referencia, que relacionen requisitos, módulos, evidencias y limitaciones. En tercer lugar, capturas reales del prototipo cuando el entorno esté estabilizado: interfaz del builder, inventario NetBox, estado de PostgreSQL, laboratorio Containerlab, artefactos Ansible y consola de nodos FRR.

Esta estrategia evita que el documento sea una narración de código. El objetivo es que cada figura o tabla ayude a defender una decisión de ingeniería o una evidencia de validación.

### 3.6. Criterios de validación

La validación se organiza por capas:

- **Modelo:** pruebas de schemas, integridad referencial, duplicados, rangos, cables, VLAN, zonas y conductos.
- **API:** endpoints de salud, topología, proyectos granulares, pipeline, runtime y cumplimiento.
- **Persistencia:** proyectos, revisiones, entidades granulares, outbox y nodos runtime.
- **Inventario:** correspondencia entre el modelo GEMEROTIC y NetBox.
- **Runtime:** correspondencia entre artefactos generados y nodos desplegados en Containerlab.
- **Automatización:** coherencia de inventario y playbooks Ansible, incluyendo syntax-check.

- **Frontend:** transformación del estado visual en modelo formal y controles operativos del builder.

La memoria final incorporará resultados de pruebas automatizadas y evidencias recogidas mediante herramientas MCP acotadas. Esta validación por capas permite sostener afirmaciones técnicas sin depender únicamente de capturas visuales.

## CAPÍTULO 4

# ARQUITECTURA DE GEMEROTIC

### 4.1. Visión general

GEMEROTIC se diseña como un constructor de gemelos digitales para redes OT/IT. La plataforma permite que el usuario modele una red industrial desde una interfaz visual, pero transforma ese diseño en una representación técnica persistente, validable y capaz de generar inventario, artefactos y runtime de laboratorio.

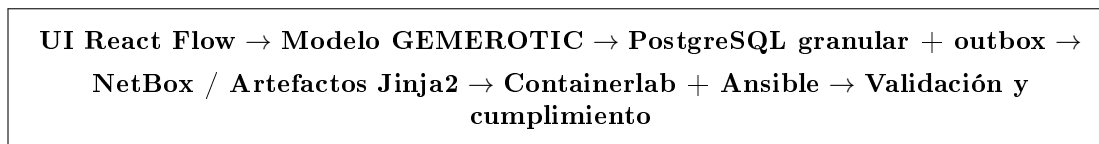


Figura 4.1: Flujo arquitectónico global de GEMEROTIC.

El elemento central no es NetBox ni el laboratorio, sino el modelo GEMEROTIC persistido de forma granular. NetBox actúa como inventario enriquecido derivado. Containerlab, Ansible, Batfish y OPA actúan como consumidores de artefactos. Esta separación permite que el gemelo sea editable, auditable y reproducible sin depender de que cada destino externo esté disponible en todo momento.

### 4.2. Principios arquitectónicos

La arquitectura se apoya en seis principios:

- **Modelo técnico antes que dibujo:** el lienzo es la interfaz de entrada, pero el resultado relevante es un modelo formal.
- **Persistencia granular:** el estado editable se almacena por entidades, revisiones y eventos, no solo como una imagen o snapshot monolítico.
- **Inventario derivado:** NetBox documenta activos y relaciones, pero no sustituye al modelo operativo del gemelo.
- **Reproducibilidad:** los artefactos se generan desde el último estado persistido mediante plantillas.
- **Validación fuera de producción:** las pruebas se desplazan a laboratorio, análisis estático o cumplimiento consultivo.
- **Evolución modular:** observabilidad, OPA avanzado, RAG normativo y perfiles runtime pueden incorporarse sin rediseñar el núcleo.

### 4.3. Modelo de datos en tres capas

El modelo de datos divide la topología en tres capas. Esta separación evita mezclar responsabilidades y permite mapear conceptos industriales con claridad.

| Capa                | Entidades                                                         | Finalidad                                                                        |
|---------------------|-------------------------------------------------------------------|----------------------------------------------------------------------------------|
| <b>Física</b>       | Sites, salas, racks, dispositivos, puertos, patch panels y cables | Representar infraestructura tangible, ubicación y terminaciones físicas.         |
| <b>Lógica</b>       | Interfaces, MAC, IPv4/IPv6 y VLAN                                 | Representar conectividad lógica y servicios asociados a puertos físicos.         |
| <b>Seguridad OT</b> | Zonas, conductos, niveles Purdue, Security Levels y criticidad    | Representar segmentación industrial y requisitos de comunicación entre dominios. |

Tabla 4.1: Modelo de datos en tres capas empleado por GEMEROTIC.

Esta estructura permite validar integridad referencial: un cable debe terminar en puertos existentes, una VLAN debe referenciar interfaces válidas, una zona debe contener dispositivos existentes y un conducto debe conectar zonas distintas. La separación también facilita explicar el sistema ante un auditor: cada dato pertenece a una capa concreta y tiene un propósito técnico definido.

## 4.4. Constructor visual de topologías

El frontend se implementa como un constructor visual basado en React Flow. La interfaz ofrece vistas diferenciadas para la capa física, lógica y de seguridad. En la vista física se trabaja con ubicaciones, mapas, racks, activos y cableado. En la vista lógica se revisan enlaces, interfaces, direcciones y VLAN. En la vista de seguridad se modelan zonas, conductos, criticidad, niveles Purdue y Security Levels.

El enfoque visual se inspira en herramientas de diseño y simulación de red, pero el objetivo no es crear un lienzo libre sin semántica. Cada activo del canvas puede transformarse en una entidad técnica. Los dibujos auxiliares, mapas o anotaciones se mantienen como estado visual separado para no contaminar el payload operacional.

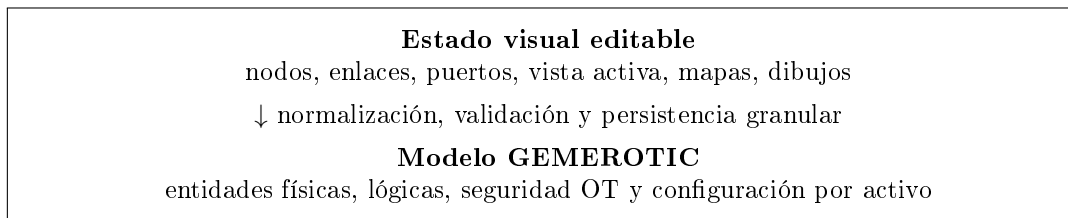


Figura 4.2: Transformación del estado visual en modelo técnico.

## 4.5. Backend, API y persistencia granular

El backend se implementa con FastAPI y Pydantic v2. FastAPI proporciona los endpoints y la documentación OpenAPI [12], mientras que Pydantic valida contratos de entrada y salida [13]. La API se organiza en superficies funcionales: salud, NetBox, topología, proyectos granulares, pipeline y cumplimiento.

La persistencia granular se implementa con PostgreSQL y SQLAlchemy. El sistema almacena proyectos, entidades de proyecto, eventos de outbox y nodos runtime. Cada guardado del builder puede traducirse a entidades como `ui.asset`, `ui.connection`, `physical.device`, `logical.interface`, `security.zone` o `asset.config`. Esta estructura permite evolucionar hacia edición granular sin perder compatibilidad con el snapshot visual de la interfaz.

El outbox desacopla el guardado del usuario de los trabajos derivados. Cuando se persiste una topología, el sistema encola eventos como sincronización de NetBox, generación de artefactos, reconciliación de despliegue y validación de cumplimiento. Si una integración falla, el estado editable no se pierde.

## 4.6. Inventario enriquecido con NetBox

NetBox se utiliza como repositorio estructurado de inventario [7]. GEMEROTIC sincroniza dispositivos, interfaces, cables, ubicaciones, racks, VLAN y metadatos relevantes. Además, un plugin específico añade conceptos OT como zonas de seguridad y conductos.

La decisión importante es que NetBox no se presenta como fuente única de verdad absoluta. En este trabajo, el estado operativo del gemelo reside en GEMEROTIC; NetBox documenta y enriquece ese estado como destino derivado. Esta elección encaja con el objetivo del proyecto: ayudar a una organización industrial a construir inventario técnico fiable sin condicionar todo el flujo a una herramienta externa.

## 4.7. Generación de artefactos

El modelo persistido se transforma en artefactos mediante plantillas Jinja2 [14]. La salida incluye topología Containerlab, inventarios y playbooks Ansible, artefactos Batfish, entradas para OPA, JSON intermedios y manifiestos. Esta separación permite revisar cada etapa y evita que el despliegue sea una caja negra.

| Destino             | Artefactos principales                                                                            |
|---------------------|---------------------------------------------------------------------------------------------------|
| <b>Containerlab</b> | Topología de nodos, enlaces, imágenes y montajes runtime.                                         |
| <b>Ansible</b>      | Inventario, variables y playbooks de configuración.                                               |
| <b>Batfish</b>      | Configuraciones y perfiles mixtos para análisis consultivo.                                       |
| <b>OPA</b>          | Entrada de evaluación y políticas Rego iniciales.                                                 |
| <b>Runtime FRR</b>  | Ficheros <code>frr.conf</code> , <code>daemons</code> y <code>vttysh.conf</code> por nodo de red. |

Tabla 4.2: Artefactos derivados del modelo GEMEROTIC.

## 4.8. Runtime transparente del gemelo

El runtime permite desplegar el gemelo con Containerlab y configurar nodos mediante Ansible. Los activos de red como routers, switches y firewalls utilizan FRR para disponer de una consola realista en la demo. Los activos OT o host genéricos se representan con imágenes Linux ligeras mientras se define una taxonomía futura de perfiles runtime por fabricante y tipo de activo.

La interfaz incorpora controles de encendido, apagado y reinicio del gemelo completo o de nodos individuales. También permite abrir terminales interactivas para activos desplegados. La sesión de terminal utiliza tokens efímeros HMAC para que la API key no viaje como parámetro persistente en el WebSocket. En nodos FRR, el sistema puede sincronizar el `running-config`, almacenarlo como

estado runtime, actualizar la configuración del activo y mantener compatibilidad con el snapshot visual.

## 4.9. Validación y cumplimiento

La validación se plantea como consultiva. Batfish se integra mediante generación de artefactos y perfiles mixtos, pero su papel no debe bloquear la existencia del gemelo. En redes legacy, el primer objetivo es poder reproducir y analizar la red; los hallazgos deben orientar mejoras, no impedir la documentación.

El motor de cumplimiento evalúa evidencias técnicas de la topología y devuelve estados como `pass`, `fail`, `warn` o `not_assessed`. La asistencia IA se sitúa encima del informe determinista: explica, prioriza y responde preguntas dentro del alcance del modelo. No certifica cumplimiento legal ni sustituye una auditoría formal.

## 4.10. Estado de implementación de la arquitectura

Tabla 4.3: Estado de implementación de los módulos principales de GEMEROTIC.

| Módulo                    | Estado               | Observaciones                                                                             |
|---------------------------|----------------------|-------------------------------------------------------------------------------------------|
| Modelo de tres capas      | Implementado         | Schemas Pydantic con validaciones referenciales entre capas.                              |
| Constructor visual        | Implementado         | UI con vistas física, lógica y seguridad; estado visual separado del modelo técnico.      |
| Persistencia granular     | Implementado         | PostgreSQL almacena proyectos, entidades, revisiones, outbox y runtime nodes.             |
| Inventario NetBox         | Implementado         | Sincronización derivada de dispositivos, interfaces, cables, VLAN, zonas y conductos.     |
| Generación de artefactos  | Implementado         | Jinja2 produce bundles para Containerlab, Ansible, Batfish, OPA y manifiestos.            |
| Runtime transparente      | Implementado         | Despliegue Containerlab/Ansible desde el último estado guardado, con controles desde UI.  |
| Terminales FRR            | Implementado         | Consola WebSocket con token efímero y sincronización de <code>running-config</code> .     |
| Batfish                   | Parcial              | Generación de artefactos y perfiles mixtos; análisis completo desde plataforma pendiente. |
| Cumplimiento determinista | Implementado inicial | Baseline técnica trazable a IEC 62443, NIS2 e ISO/IEC 27001.                              |
| Asistencia IA             | Implementado inicial | Capa explicativa local/Ollama sobre informe determinista.                                 |
| Observabilidad            | Futuro               | Integración prevista con LibreNMS y Oxidized.                                             |

CAPÍTULO 4. ARQUITECTURA DE GENERACIÓN DE PROYECTOS DE IMPLEMENTACIÓN DE LA ARQUITECTURA

| Módulo                 | Estado | Observaciones                                                 |
|------------------------|--------|---------------------------------------------------------------|
| Auditoría OPA avanzada | Futuro | Políticas de cumplimiento completas pendientes de maduración. |

## 5.1. Estructura del repositorio

El repositorio de GEMEROTIC se organiza en bloques diferenciados: backend FastAPI, frontend React, plugin NetBox, plantillas Jinja2, configuración Docker, pruebas automatizadas y documentación. Esta estructura responde a la arquitectura modular descrita en el capítulo anterior. El backend concentra la validación, persistencia, generación de artefactos y endpoints. El frontend proporciona la experiencia de construcción visual. El plugin NetBox añade semántica OT al inventario. Las plantillas y servicios de pipeline transforman el modelo en laboratorio reproducible.

## 5.2. Implementación del modelo de datos

El modelo se implementa mediante schemas Pydantic v2. La capa física define sites, salas, racks, dispositivos, puertos, patch panels y cables. La capa lógica define interfaces, direccionamiento, MAC y VLAN. La capa de seguridad OT define zonas, conductos, niveles Purdue, Security Levels y criticidad. El schema raíz valida referencias cruzadas para impedir inconsistencias antes de persistir o generar artefactos.

Una parte relevante de la implementación es la traducción desde el estado visual del builder hacia entidades granulares. El importador convierte nodos, enlaces, dibujos, layout físico, interfaces, VLAN, zonas, conductos y configuración de activos en registros diferenciados. Esta decisión permite mantener compatibilidad con la UI actual mientras se prepara una evolución hacia comandos granulares.

### 5.3. Implementación del frontend

El frontend ofrece un catálogo de activos OT/IT y un lienzo de edición con vistas física, lógica y de seguridad. El usuario puede crear equipos, definir puertos, enlazar activos, dibujar zonas o áreas de planta, editar direccionamiento, asociar VLAN, asignar criticidad y configurar niveles Purdue o Security Levels. La experiencia se orienta a operadores y responsables técnicos que necesitan entender la red desde varios planos.

La UI también incorpora controles runtime: inspección del laboratorio, encendido, apagado y reinicio de nodos, apertura de consolas y sincronización de configuración. Esta capacidad acerca el prototipo a herramientas de simulación de red, pero mantiene una frontera clara entre estado visual, modelo técnico y runtime desplegado.

### 5.4. Implementación del backend

El backend expone endpoints para salud, bootstrap de NetBox, persistencia del estado de topología, snapshot granular, comandos de proyecto, jobs, generación de artefactos, despliegue, power control, terminales runtime y cumplimiento. La lógica de negocio se divide en servicios especializados: cliente NetBox, importadores, store granular, generador de artefactos, runner del pipeline y motor de compliance.

La persistencia granular se apoya en cuatro tablas principales: proyectos, entidades, eventos de outbox y nodos runtime. Esta estructura permite distinguir estado editable, trabajos derivados y estado de ejecución. El worker procesa eventos pendientes de forma independiente, evitando que el guardado del usuario dependa de tareas potencialmente lentas.

### 5.5. Implementación del pipeline

El pipeline transforma el modelo persistido en un bundle inspeccionable. Los artefactos se separan por dominio: topología, canvas, inventario, runtime, Ansible, configuraciones y políticas. Containerlab define nodos y enlaces; Ansible aplica configuración; Batfish recibe configuraciones para análisis consultivo; OPA queda preparado para políticas de cumplimiento.

Para los nodos de red se generan ficheros FRR que permiten disponer de consola y configuración tangible. La plantilla prioriza el **running-config** sincronizado desde el nodo; si no existe, genera una configuración base desde la declaración del activo. Esta decisión permite que el laboratorio evolucione desde el diseño hacia una configuración observada.

## 5.6. Implementación del cumplimiento

El motor de cumplimiento genera informes estructurados con controles, estados, severidad, evidencias y recomendaciones. Los controles iniciales se relacionan con segmentación, zonas, conductos, criticidad, niveles de seguridad e interfaces de gestión. El asistente IA utiliza el informe como contexto cerrado y no altera la decisión del motor determinista.

Esta separación es importante desde el punto de vista académico: el prototipo puede apoyarse en IA para explicar resultados, pero no delega en ella el juicio de cumplimiento. La validación normativa debe seguir siendo trazable y auditable.

## 5.7. Despliegue y entorno de ejecución

El entorno de ejecución contempla API, UI, PostgreSQL GEMEROTIC, NetBox con plugin OT, Valkey, worker de outbox, Docker, Containerlab, Ansible y, opcionalmente, Ollama. En servidor, el API se ejecuta con acceso controlado al Docker del host para desplegar laboratorios. En local, la documentación de arranque permite levantar los servicios principales para desarrollo y pruebas.

La diferencia entre edición local y despliegue en servidor debe explicarse con claridad: el entorno de edición permite revisar y modificar la memoria o el código, mientras que el servidor valida el comportamiento del gemelo desplegado con herramientas reales.

## 6.1. Estrategia de pruebas

La validación del prototipo combina pruebas automatizadas, compilación de frontend, comprobaciones de backend y evidencias del entorno desplegado. La estrategia se inspira en el enfoque de los TFM de referencia: no basta con describir la arquitectura, sino que debe demostrarse mediante resultados intermedios, tablas y capturas verificables.

Las pruebas automatizadas cubren schemas, endpoints, seguridad, rate limiting, importadores, generación de artefactos, pipeline, persistencia granular, frontend y cliente API. Además, el entorno runtime se valida mediante herramientas MCP de solo lectura que resumen el estado de PostgreSQL, NetBox, Containerlab, Ansible y Docker.

## 6.2. Validación del modelo de datos

La validación del modelo comprueba que una topología cumple las reglas de integridad antes de generar inventario o artefactos. Entre los casos verificados se encuentran identificadores duplicados, referencias inexistentes, cables con puertos reutilizados, VLAN fuera de rango, conductos inválidos, zonas sin dispositivos existentes y formatos incorrectos de direcciones o MAC.

Esta validación es esencial para que el constructor visual no produzca un simple dibujo. Cada operación debe terminar en una estructura técnica coherente. De este modo, el gemelo puede utilizarse como entrada fiable para inventario, despliegue y análisis.

### 6.3. Validación del constructor visual

El constructor visual se valida comprobando que un estado de nodos, enlaces, puertos y atributos se transforma correctamente en entidades técnicas. La UI mantiene separados los elementos gráficos auxiliares del payload operacional. Esta separación impide que anotaciones o dibujos de planta interfieran con la lógica de red, pero permite conservarlos como contexto visual del proyecto.

En la memoria final se incorporarán capturas de las tres vistas principales: física, lógica y seguridad. También se documentará el panel de payload o estado técnico para mostrar la relación entre la edición visual y el modelo generado.

### 6.4. Validación de persistencia e inventario

El proyecto de validación principal, denominado **nuevo-proyecto-ot**, muestra un estado granular consistente. La base PostgreSQL contiene el proyecto en revisión 5, con estado de NetBox sincronizado, despliegue listo y validación lista. El recuento granular incluye 15 activos visuales, 14 conexiones, 15 dispositivos físicos, 64 interfaces lógicas, 4 VLAN, 4 zonas de seguridad y 6 conductos.

NetBox refleja el mismo alcance de inventario: 15 dispositivos, 64 interfaces, 14 cables, 1 site, 1 location, 1 rack, 4 VLAN, 4 zonas OT y 6 conductos. Esta correspondencia demuestra que NetBox actúa como inventario derivado coherente con el estado persistido por GEMEROTIC.

| Evidencia                  | Resultado                                 | Interpretación                                                       |
|----------------------------|-------------------------------------------|----------------------------------------------------------------------|
| <b>PostgreSQL granular</b> | 15 activos, 14 conexiones                 | El estado editable está persistido por entidades.                    |
| <b>NetBox</b>              | 15 dispositivos, 64 interfaces, 14 cables | El inventario derivado coincide con el modelo.                       |
| <b>Zonas y conductos</b>   | 4 zonas, 6 conductos                      | La semántica OT se conserva fuera del lienzo visual.                 |
| <b>Outbox</b>              | Eventos derivados completados             | Sincronización, artefactos, despliegue y validación quedan trazados. |

Tabla 6.1: Evidencias de persistencia e inventario para el proyecto de validación.

### 6.5. Validación del pipeline y runtime

El laboratorio **nuevo-proyecto-ot** despliega 15 nodos y 14 enlaces en Containerlab. La distribución de imágenes incluye 8 nodos FRR para activos de red y 7 nodos Alpine para hosts o activos

OT genéricos. La topología generada coincide con el runtime en ejecución: no hay nodos esperados ausentes ni nodos extra.

Ansible valida un inventario de 15 nodos y 28 interfaces, sin identificadores duplicados ni contenedores ausentes. El playbook generado supera el **syntax-check**. Estas evidencias permiten afirmar que el pipeline no solo produce archivos, sino que genera artefactos coherentes con un laboratorio desplegable.

## 6.6. Validación de cumplimiento

La capa de cumplimiento se valida mediante controles deterministas sobre la topología. Los resultados se clasifican como **pass**, **fail**, **warn** o **not\_assessed**, acompañados de evidencias. En la versión actual, el objetivo es demostrar trazabilidad y utilidad técnica, no emitir una certificación legal.

La asistencia IA se evalúa como capa de explicación. Su entrada debe ser el informe ya calculado, y sus respuestas deben mantenerse dentro del alcance de la topología y de la baseline modelada. Esta restricción evita que la IA invente controles o sustituya el criterio experto.

## 6.7. Resultados obtenidos

Los resultados actuales permiten defender la viabilidad del enfoque:

- El prototipo mantiene un modelo persistente y granular del gemelo.
- El inventario derivado en NetBox coincide con el estado del proyecto.
- El pipeline genera artefactos coherentes para Containerlab y Ansible.
- El runtime despliega nodos y enlaces acordes con la topología.
- Los nodos FRR proporcionan consola tangible para activos de red.
- El sistema conserva el trabajo del usuario aunque las integraciones derivadas sean asíncronas.

En una iteración posterior se incorporarán capturas de UI, NetBox, runtime, consola y artefactos para completar la evidencia visual.

## 6.8. Limitaciones observadas

Las limitaciones actuales se concentran en cuatro ámbitos. Primero, Batfish está integrado como generación de artefactos y validación consultiva pendiente de ejecución completa desde la plataforma. Segundo, OPA no se utiliza todavía como motor obligatorio de auditoría avanzada. Tercero, el RAG normativo completo queda como línea de evolución. Cuarto, los activos OT genéricos no se representan aún con imágenes de fabricante o perfiles runtime específicos.

Estas limitaciones no invalidan el prototipo: delimitan el estado alcanzado y justifican el trabajo futuro. En un TFM de ingeniería aplicada, esta separación entre arquitectura objetivo, prototipo implementado y evolución prevista aporta rigor y evita sobredimensionar los resultados.

## CAPÍTULO 7

# ANÁLISIS DEL VALOR Y APLICABILIDAD INDUSTRIAL

### 7.1. Valor para redes OT reales

El valor principal de GEMEROTIC se encuentra en reducir el riesgo de actuar sobre redes OT sin una representación técnica suficiente. En organizaciones industriales con varias plantas, activos heterogéneos y continuidad productiva crítica, el inventario incompleto y la falta de trazabilidad dificultan responder preguntas básicas: qué equipos existen, cómo están conectados, qué zonas comunican entre sí, qué cambios se han ensayado y qué evidencias respaldan una decisión.

Un gemelo digital no elimina la necesidad de auditoría, pero puede reducir el coste y el riesgo de prepararla. Permite modelar la red, reproducir escenarios, revisar cambios y conservar evidencias sin interrumpir la operación real. Para una organización como AMC Global, este enfoque puede apoyar procesos de ciber-resiliencia, mejora continua y preparación ante marcos regulatorios.

### 7.2. Comparación frente a enfoques tradicionales

Frente a inventarios manuales o diagramas estáticos, GEMEROTIC propone un modelo persistente y transformable. Frente a una automatización directa, prioriza una fase previa de representación, validación y documentación. Esta diferencia es clave en OT: el objetivo inicial no es cambiar la red más rápido, sino entenderla mejor y reducir el riesgo de cualquier intervención.

El enfoque también se diferencia de una simulación aislada. El gemelo no se limita a levantar nodos; conecta inventario, topología, seguridad OT, artefactos y evidencias. Esa integración permite

que el modelo sirva tanto para ingeniería de red como para ciberseguridad y auditoría técnica.

### 7.3. Aplicabilidad a sectores críticos

Aunque el caso motivador se sitúa en el sector alimentario y de bebidas, el enfoque es aplicable a otros sectores con redes OT: energía, agua, manufactura, transporte, logística, farmacéutica o infraestructuras públicas. En todos ellos aparecen restricciones similares: procesos continuos, ventanas de mantenimiento limitadas, activos legacy, exigencias regulatorias y necesidad de justificar decisiones técnicas.

La arquitectura modular permite adaptar el catálogo de activos, los perfiles runtime y los controles de cumplimiento a cada sector. En una planta alimentaria pueden priorizarse líneas de producción, SCADA, PLC y redes de planta; en energía pueden aparecer subestaciones, relés, telecontrol y protocolos específicos; en agua pueden destacarse estaciones remotas, sensores y telemetría.

### 7.4. Encaje con estándares y auditorías

GEMEROTIC no pretende certificar automáticamente el cumplimiento de ISA/IEC 62443, NIS2, NIST SP 800-82 o ENS. Su aportación consiste en estructurar evidencias técnicas que ayuden a una organización a prepararse para auditorías, revisiones internas y procesos de mejora continua.

El modelo de zonas y conductos facilita explicar segmentación. El inventario derivado ayuda a documentar activos. La persistencia granular permite conservar revisiones. Los artefactos reproducibles permiten justificar qué laboratorio o configuración se ha evaluado. El compliance determinista aporta hallazgos trazables. En conjunto, la plataforma puede convertirse en una herramienta de apoyo para responsables de seguridad, ingeniería y operación.

### 7.5. Evolución empresarial

Desde una perspectiva de producto, GEMEROTIC puede evolucionar hacia una plataforma de documentación, validación y asistencia a la auditoría para redes OT. Su valor diferencial reside en combinar un constructor visual, un modelo formal, inventario enriquecido, despliegue de laboratorio, validación consultiva y semántica normativa industrial.

La colaboración con una empresa industrial permite orientar la evolución hacia necesidades reales: integración con inventarios existentes, plantillas de planta, capturas de evidencias, perfiles

por tipo de activo, validación de cambios y cuadros de mando de ciber-resiliencia. Esta orientación evita desarrollar una herramienta técnicamente interesante pero desconectada del uso operativo.

### 8.1. Conclusiones técnicas

El trabajo demuestra la viabilidad de plantear un constructor de gemelos digitales para redes OT/IT a partir de un modelo formal, una interfaz visual y un pipeline de artefactos reproducibles. La arquitectura propuesta permite separar estado visual, modelo técnico, inventario derivado y runtime de laboratorio. Esta separación es fundamental para evitar que el sistema sea solo una herramienta de dibujo o una automatización rígida.

La persistencia granular y el out box representan una evolución relevante del prototipo. Permiten guardar el trabajo del usuario, desacoplar integraciones externas y procesar tareas derivadas sin bloquear la edición. Además, el uso de Containerlab, Ansible y FRR muestra que el gemelo puede pasar de modelo a laboratorio interactivo.

### 8.2. Conclusiones de ciberseguridad OT

Desde la perspectiva de ciberseguridad OT, el enfoque aporta valor porque permite analizar sin intervenir directamente sobre producción. La plataforma facilita estructurar inventario, segmentación, comunicaciones, criticidad y evidencias, elementos necesarios para avanzar en ciber-resiliencia y conformidad con marcos como ISA/IEC 62443, NIS2, NIST SP 800-82 y ENS.

La colaboración con AMC Global refuerza la motivación industrial del trabajo. El problema no es abstracto: las organizaciones con varias plantas y procesos críticos necesitan herramientas que permitan comprender y validar sus redes antes de ejecutar cambios que puedan afectar a la

continuidad operacional.

### 8.3. Limitaciones del prototipo

El prototipo no sustituye una auditoría formal ni certifica cumplimiento legal. Tampoco incorpora todavía observabilidad completa, análisis Batfish ejecutado de extremo a extremo desde la plataforma, auditoría OPA avanzada, perfiles runtime específicos por fabricante ni RAG normativo con trazabilidad completa de fuentes.

Estas limitaciones delimitan el alcance desarrollado. La aportación principal del TFM no es cerrar todas las capacidades de una plataforma industrial madura, sino diseñar una arquitectura viable, implementar sus bases y validar que el enfoque puede generar inventario, artefactos y evidencias útiles.

### 8.4. Trabajo futuro

Las líneas de trabajo futuro incluyen:

- Integración de observabilidad con LibreNMS y Oxidized para comparar estado diseñado, desplegado y observado.
- Auditoría avanzada de cumplimiento mediante OPA y políticas Rego más completas.
- Ejecución completa de análisis Batfish desde la plataforma, con consultas de reachability, aislamiento y cambios de configuración.
- Incorporación de un sistema RAG normativo con fuentes controladas, citas trazables y separación clara entre evidencia, recomendación y explicación.
- Definición de runtime profiles por tipo de activo OT, fabricante, licencia y función.
- Gestión multiusuario, autenticación avanzada, gobierno de proyectos y control de permisos por rol.
- Validación con escenarios industriales reales o pilotos controlados en colaboración con empresas.
- Preparación de una metodología de adopción para organizaciones con inventario incompleto o redes legacy.

---

## BIBLIOGRAFÍA

- [1] AMC Global. *AMC Global*. URL: <https://www.amcglobalco.com/es/> (visitado 01-06-2026).
- [2] AMC Natural. *Nosotros*. URL: <https://amcnatural.com/nosotros/> (visitado 01-06-2026).
- [3] National Institute of Standards and Technology. *NIST SP 800-82 Rev. 3: Guide to Operational Technology (OT) Security*. URL: <https://csrc.nist.gov/pubs/sp/800/82/r3/final> (visitado 01-06-2026).
- [4] International Society of Automation. *ISA/IEC 62443 Series of Standards*. URL: <https://www.isa.org/standards-and-publications/isa-standards/isa-iec-62443-series-of-standards> (visitado 08-05-2026).
- [5] European Parliament and Council of the European Union. *Directive (EU) 2022/2555 on measures for a high common level of cybersecurity across the Union*. URL: <https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX:32022L2555> (visitado 08-05-2026).
- [6] Boletín Oficial del Estado. *Real Decreto 311/2022, de 3 de mayo, por el que se regula el Esquema Nacional de Seguridad*. URL: <https://www.boe.es/eli/es/rd/2022/05/03/311> (visitado 01-06-2026).
- [7] NetBox Labs. *NetBox Documentation*. URL: <https://netboxlabs.com/docs/netbox/en/stable/> (visitado 08-05-2026).
- [8] Containerlab. *Containerlab Documentation*. URL: <https://containerlab.dev/> (visitado 08-05-2026).
- [9] Ansible Community. *Ansible Documentation*. URL: <https://docs.ansible.com/projects/ansible/latest/> (visitado 08-05-2026).
- [10] Batfish. *Batfish: An open source network configuration analysis tool*. URL: <https://batfish.org/> (visitado 08-05-2026).
- [11] Open Policy Agent. *Open Policy Agent Documentation*. URL: <https://www.openpolicyagent.org/docs/latest> (visitado 08-05-2026).
- [12] FastAPI. *FastAPI Documentation*. URL: <https://fastapi.tiangolo.com/> (visitado 08-05-2026).

- [13] Pydantic. *Pydantic Documentation*. URL: <https://pydantic.dev/> (visitado 08-05-2026).
- [14] Pallets Projects. *Jinja Documentation*. URL: <https://jinja.palletsprojects.com/> (visitado 08-05-2026).
- [15] xyflow. *React Flow Documentation*. URL: <https://reactflow.dev/> (visitado 08-05-2026).

# Anexos

# ANEXO A

---

## MANUAL DE INSTALACIÓN Y DESPLIEGUE

Este anexo recogerá los pasos necesarios para levantar el entorno completo del prototipo: Net-Box, PostgreSQL, Valkey, API, UI, plugin OT, herramientas de pipeline y asistente IA. La base operativa se encuentra actualmente documentada en el repositorio y se consolidará aquí cuando el flujo final quede estabilizado.

## ANEXO B

### EJEMPLOS DE PAYLOAD TOPOLOGYCREATE

Este anexo incluirá ejemplos representativos del contrato `TopologyCreate`, mostrando activos, interfaces, VLAN, zonas, conductos y canvas.

## ANEXO C ARTEFACTOS GENERADOS

Este anexo recogerá ejemplos de artefactos generados por el pipeline: topología Containerlab, inventario Ansible, variables, configuraciones Batfish y entrada de cumplimiento.

## ANEXO D

### CAPTURAS DE INTERFAZ Y RESULTADOS

Este anexo se reservará para capturas de la UI, inventario en NetBox, ejecución del pipeline, informes de cumplimiento y resultados de validación.

## ANEXO E

CÓDIGO DE INTERÉS

Este anexo incluirá fragmentos relevantes de código cuando el prototipo quede estabilizado, priorizando aquellos que expliquen decisiones arquitectónicas: transformación del estado visual, validación del modelo, generación de artefactos y motor de cumplimiento.